

SHETH L.U.J & SIR MV COLLEGE

Practical 1: Breadth First Search & Iterative Depth First Search in Python.

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

#GRAPH

```
course_prerequisites = {
    'Intro to Programming': ['Data Structures', 'Web Development'],
    'Data Structures': ['Algorithms', 'Artificial Intelligence'],
    'Web Development': [],
    'Algorithms': [],
    'Artificial Intelligence': []
}
```

#BFS

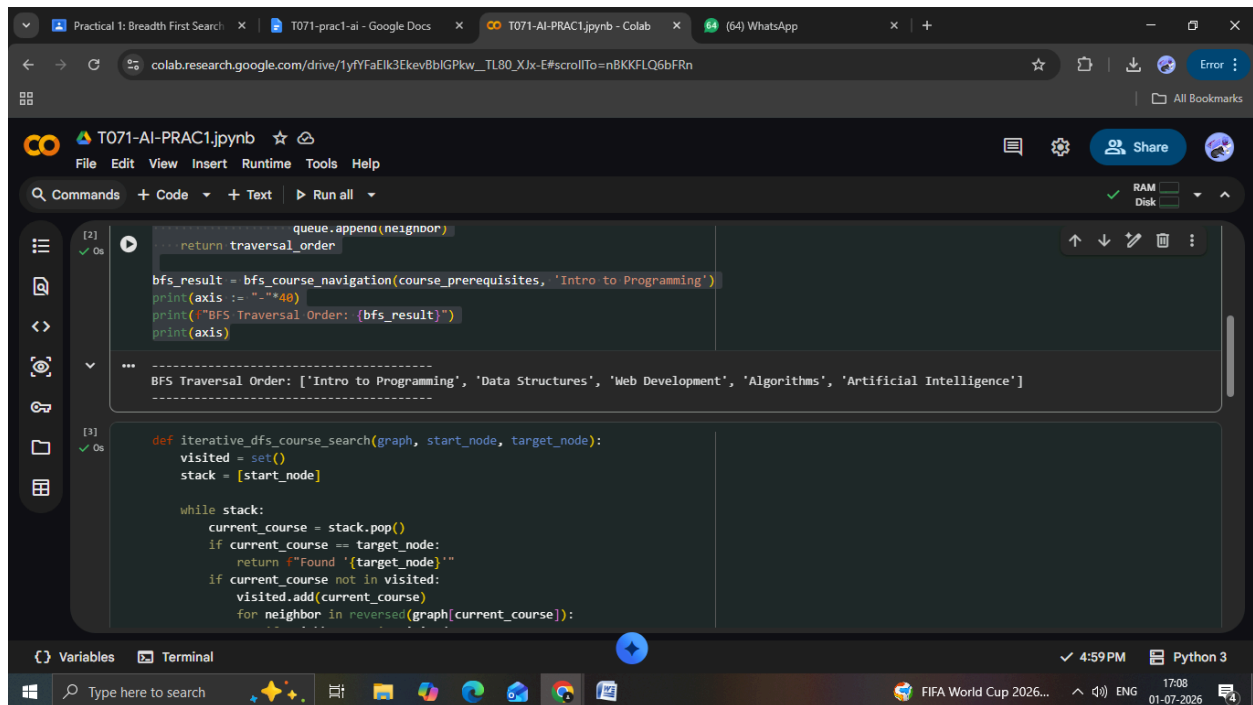
```
from collections import deque
def bfs_course_navigation(graph, start_node):
    visited = set()
    queue = deque([start_node])
    traversal_order = []

    while queue:
        current_course = queue.popleft()
        if current_course not in visited:
            visited.add(current_course)
            traversal_order.append(current_course)
            for neighbor in graph[current_course]:
                if neighbor not in visited:
                    queue.append(neighbor)
    return traversal_order

bfs_result = bfs_course_navigation(course_prerequisites, 'Intro to
Programming')
print(axis := "-"*40)
print(f"BFS Traversal Order: {bfs_result}")
print(axis)
```

SHETH L.U.J & SIR MV COLLEGE

#OUTPUT



The screenshot shows a Google Colab notebook with two code cells. The first cell contains a BFS function and its output, and the second cell contains a DFS function. The output of the DFS function is shown in the terminal.

```
[2] In [2]: queue.append(neighbour)
           return traversal_order

bfs_result = bfs_course_navigation(course_prerequisites, 'Intro to Programming')
print(axis := "--*48")
print(f"BFS Traversal Order: {bfs_result}")
print(axis)

BFS Traversal Order: ['Intro to Programming', 'Data Structures', 'Web Development', 'Algorithms', 'Artificial Intelligence']

[3] In [3]: def iterative_dfs_course_search(graph, start_node, target_node):
           visited = set()
           stack = [start_node]

           while stack:
               current_course = stack.pop()
               if current_course == target_node:
                   return f"Found '{target_node}'"
               if current_course not in visited:
                   visited.add(current_course)
                   for neighbor in reversed(graph[current_course]):
                       if neighbor not in visited:
                           stack.append(neighbor)
           return f"'{target_node}' not reachable"

dfs_result = iterative_dfs_course_search(course_prerequisites, 'Intro to Programming', 'Artificial Intelligence')
print(f"DFS Search Result: {dfs_result}")
```

#DFS

```
def iterative_dfs_course_search(graph, start_node, target_node):
    visited = set()
    stack = [start_node]

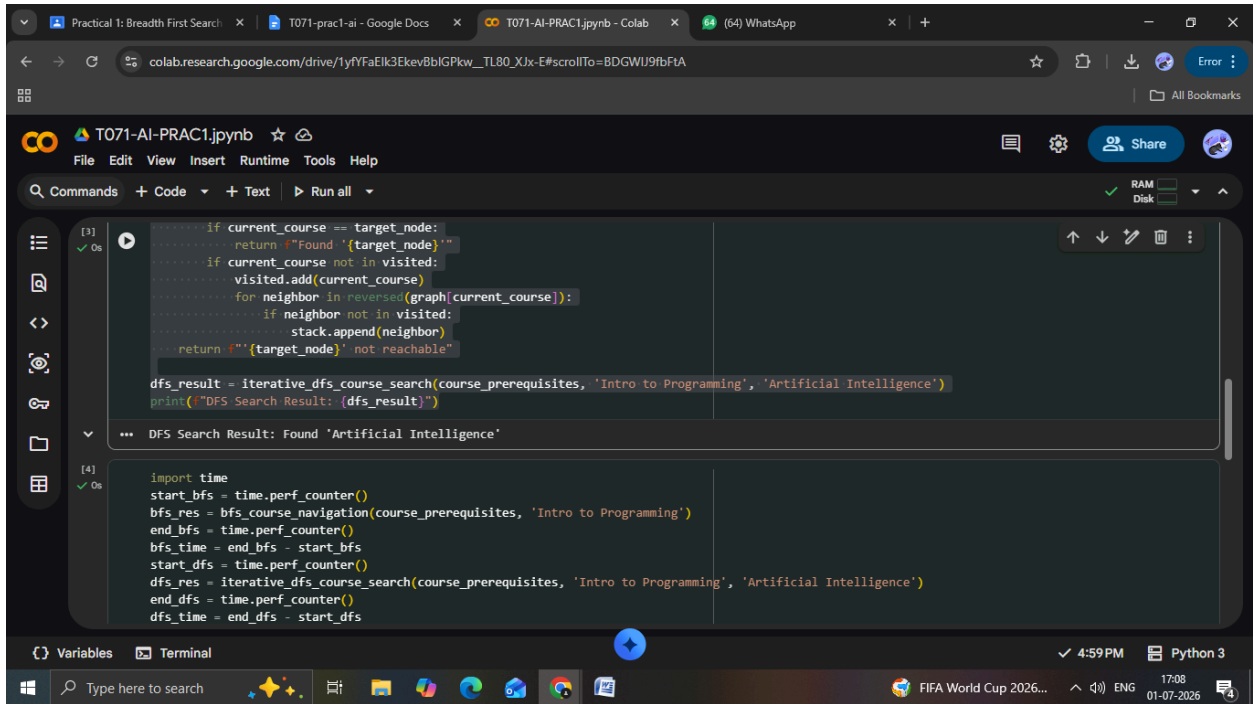
    while stack:
        current_course = stack.pop()
        if current_course == target_node:
            return f"Found '{target_node}'"
        if current_course not in visited:
            visited.add(current_course)
            for neighbor in reversed(graph[current_course]):
                if neighbor not in visited:
                    stack.append(neighbor)
    return f"'{target_node}' not reachable"

dfs_result = iterative_dfs_course_search(course_prerequisites, 'Intro to Programming', 'Artificial Intelligence')
print(f"DFS Search Result: {dfs_result}")
```

#OUTPUT

Gazala alam
T071

SHETH L.U.J & SIR MV COLLEGE



The screenshot shows a Google Colab notebook titled 'T071-AI-PRAC1.jpynb'. The notebook contains two code cells. The first cell implements a Depth-First Search (DFS) algorithm to find a target node in a graph of course prerequisites. The second cell implements a Breadth-First Search (BFS) algorithm for the same purpose. The output of the first cell shows the DFS result: 'Found 'Artificial Intelligence''. The output of the second cell shows the BFS result: 'Found 'Artificial Intelligence''. The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for commands, code, text, and running, and a status bar at the bottom showing the time (4:59 PM) and Python version (3).

```
[1] ✓ Os
def iterative_dfs_course_search(course_prerequisites, 'Intro to Programming', 'Artificial Intelligence'):
    if current_course == target_node:
        return f"Found '{target_node}'"
    if current_course not in visited:
        visited.add(current_course)
        for neighbor in reversed(graph[current_course]):
            if neighbor not in visited:
                stack.append(neighbor)
                return f"Found '{target_node}'"
    return f"'{target_node}' not reachable"

dfs_result = iterative_dfs_course_search(course_prerequisites, 'Intro to Programming', 'Artificial Intelligence')
print(f"DFS Search Result: {dfs_result}")

... DFS Search Result: Found 'Artificial Intelligence'

[4] ✓ Os
import time
start_bfs = time.perf_counter()
bfs_res = bfs_course_navigation(course_prerequisites, 'Intro to Programming')
end_bfs = time.perf_counter()
bfs_time = end_bfs - start_bfs
start_dfs = time.perf_counter()
dfs_res = iterative_dfs_course_search(course_prerequisites, 'Intro to Programming', 'Artificial Intelligence')
end_dfs = time.perf_counter()
dfs_time = end_dfs - start_dfs

print("PERFORMANCE & EFFICIENCY COMPARISON REPORT")
print(f"BFS Execution Time : {bfs_time:.7f} seconds")
print(f"DFS Execution Time : {dfs_time:.7f} seconds")
print("Memory Efficiency Context:")
print("- BFS Space Complexity : O(b^d) -> High memory (stores full levels in queue)")
print("- DFS Space Complexity : O(bm) -> Low memory (stores only current path branch on stack)")
print("Operational Behavior Summary:")
```

#COMPARISION

```
import time
start_bfs = time.perf_counter()
bfs_res = bfs_course_navigation(course_prerequisites, 'Intro to Programming')
end_bfs = time.perf_counter()
bfs_time = end_bfs - start_bfs
start_dfs = time.perf_counter()
dfs_res = iterative_dfs_course_search(course_prerequisites, 'Intro to Programming', 'Artificial Intelligence')
end_dfs = time.perf_counter()
dfs_time = end_dfs - start_dfs

print("PERFORMANCE & EFFICIENCY COMPARISON REPORT")
print(f"BFS Execution Time : {bfs_time:.7f} seconds")
print(f"DFS Execution Time : {dfs_time:.7f} seconds")
print("Memory Efficiency Context:")
print("- BFS Space Complexity : O(b^d) -> High memory (stores full levels in queue)")
print("- DFS Space Complexity : O(bm) -> Low memory (stores only current path branch on stack)")
print("Operational Behavior Summary:")
```

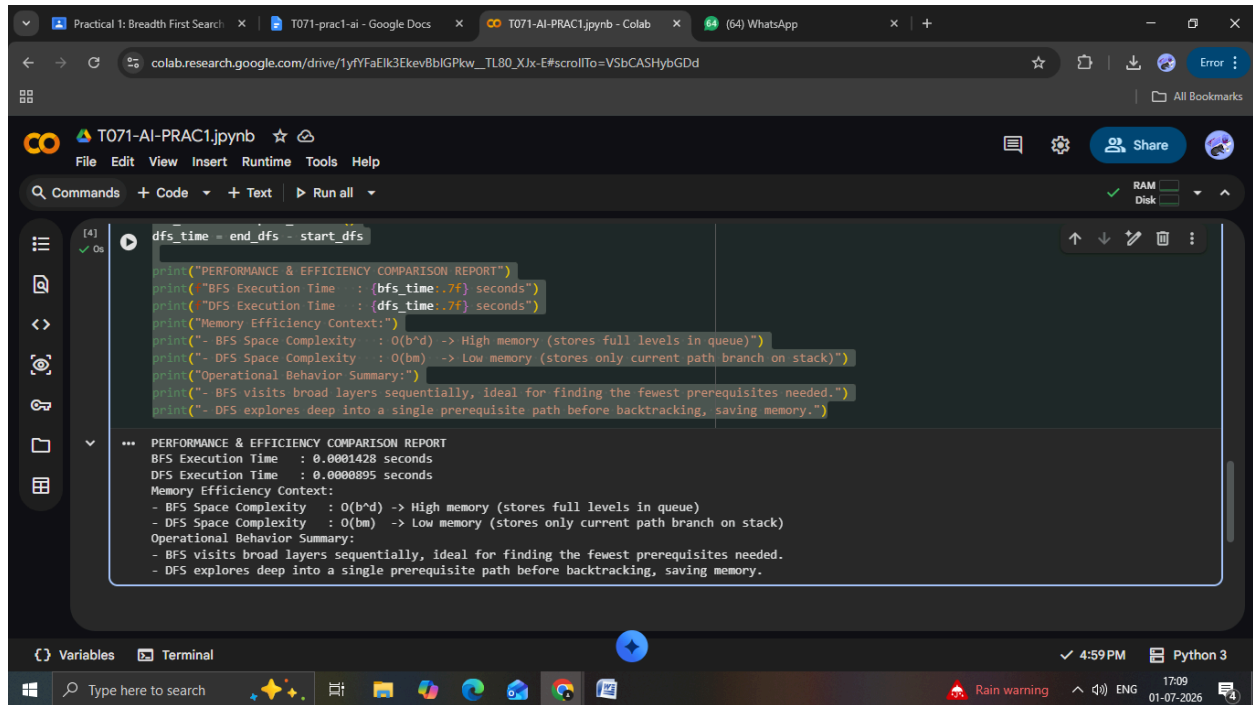
Gazala alam

T071

SHETH L.U.J & SIR MV COLLEGE

```
print("- BFS visits broad layers sequentially, ideal for finding the  
fewest prerequisites needed.")  
print("- DFS explores deep into a single prerequisite path before  
backtracking, saving memory.")
```

#OUTPUT



The screenshot shows a Google Colab notebook interface. The top bar includes tabs for 'Practical 1: Breadth First Search', 'T071-prac1-ai - Google Docs', and 'T071-AI-PRAC1.jpynb - Colab'. The notebook title is 'T071-AI-PRAC1.jpynb'. The code cell contains the following Python code:

```
dfs_time = end_dfs - start_dfs  
  
print("PERFORMANCE & EFFICIENCY COMPARISON REPORT")  
print("BFS Execution Time : {bfs_time:.7f} seconds")  
print("DFS Execution Time : {dfs_time:.7f} seconds")  
print("Memory Efficiency Context:")  
print("- BFS Space Complexity : O(b*d) -> High memory (stores full levels in queue)")  
print("- DFS Space Complexity : O(bm) -> Low memory (stores only current path branch on stack)")  
print("Operational Behavior Summary:")  
print("- BFS visits broad layers sequentially, ideal for finding the fewest prerequisites needed.")  
print("- DFS explores deep into a single prerequisite path before backtracking, saving memory.")
```

The output of the code is displayed below the code cell:

```
PERFORMANCE & EFFICIENCY COMPARISON REPORT  
BFS Execution Time : 0.0001428 seconds  
DFS Execution Time : 0.0000895 seconds  
Memory Efficiency Context:  
- BFS Space Complexity : O(b*d) -> High memory (stores full levels in queue)  
- DFS Space Complexity : O(bm) -> Low memory (stores only current path branch on stack)  
Operational Behavior Summary:  
- BFS visits broad layers sequentially, ideal for finding the fewest prerequisites needed.  
- DFS explores deep into a single prerequisite path before backtracking, saving memory.
```

The bottom of the screenshot shows the Windows taskbar with the time 4:59 PM and the date 01-07-2026.